ЛАБОРАТОРНАЯ РАБОТА № 2. РАЗРАБОТКА МОДУЛЬНЫХ И ИНТЕГРАЦИОННЫХ ТЕСТОВ

Цель: изучить принципы разработки модульных и интеграционных тестов.

План занятия:

1. Изучить теоретические сведения.
2. Выполнить практическое задание по лабораторной работе.
3. Оформить отчёт и ответить на контрольные вопросы.

Теоретические сведения

Модульное тестирование

Модульное тестирование — это тестирование программы на уровне отдельно взятых модулей, функций или классов. Цель модульного тестирования состоит в выявлении локализованных в модуле ошибок в реализации алгоритмов, а также в определении степени готовности системы к переходу на следующий уровень разработки и тестирования. Модульное тестирование проводится по принципу «белого ящика», то есть основывается на знании внутренней структуры программы, и часто включает те или иные методы анализа покрытия кода.

Модульное тестирование обычно подразумевает создание вокруг каждого модуля определённой среды, включающей заглушки для всех интерфейсов тестируемого модуля. Некоторые из них могут использоваться для подачи входных значений, другие — для анализа результатов, присутствие третьих может быть продиктовано требованиями, накладываемыми компилятором и сборщиком.

На уровне модульного тестирования проще всего обнаружить дефекты, связанные с алгоритмическими ошибками и ошибками кодирования алгоритмов, типа работы с условиями и счётчиками циклов, а также с использованием локальных переменных и ресурсов. Ошибки, связанные с неверной трактовкой данных, некорректной реализацией интерфейсов, совместимостью, производительностью и т. п. обычно пропускаются на уровне модульного тестирования и выявляются на более поздних стадиях тестирования.

Именно эффективность обнаружения тех или иных типов дефектов должна определять стратегию модульного тестирования, то есть расстановку акцентов при определении набора входных значений. У организации, занимающейся разработкой программного обеспечения, как правило, имеется историческая база данных (Repository) раз-

работок, хранящая конкретные сведения о разработке предыдущих проектов: о версиях и сборках кода (build), зафиксированных в процессе разработки продукта, о принятых решениях, допущенных просчётах, ошибках, успехах и т. п. Проведя анализ характеристик прежних проектов, подобных заказанному организации, можно предохранить новую разработку от старых ошибок, например определив типы дефектов, поиск которых наиболее эффективен на различных этапах тестирования.

В данном случае анализируется этап модульного тестирования. Если анализ не дал нужной информации, например в случае проектов, в которых соответствующие данные не собирались, то основным правилом становится поиск локальных дефектов, у которых код, ресурсы и информация, вовлечённые в дефект, характерны именно для данного модуля. В этом случае на модульном уровне ошибки, связанные, например, с неверным порядком или форматом параметров модуля, могут быть пропущены, поскольку они вовлекают информацию, затрагивающую другие модули (а именно: спецификацию интерфейса), в то время как ошибки в алгоритме обработки параметров довольно легко обнаруживаются.

Являясь по способу исполнения структурным тестированием или тестированием «белого ящика», модульное тестирование характеризуется степенью, в которой тесты выполняют или покрывают логику программы (исходный текст).

Интеграционное тестирование

Интеграционное тестирование — это тестирование части системы, состоящей из двух и более модулей. Основная задача интеграционного тестирования — поиск дефектов, связанных с ошибками в реализации и интерпретации интерфейсного взаимодействия между модулями.

С технологической точки зрения интеграционное тестирование является количественным развитием модульного, поскольку так же, как и модульное тестирование, оперирует интерфейсами модулей и подсистем и требует создания тестового окружения, включая заглушки (Stub) на месте отсутствующих модулей. Основная разница между модульным и интеграционным тестированием состоит в целях, то есть в типах обнаруживаемых дефектов, которые в свою очередь определяют стратегию выбора входных данных и методов анализа. В частности, на уровне интеграционного тестирования часто применяются методы, связанные с покрытием интерфейсов, например вызовов функций или методов, или анализ использования интер-

фейсных объектов, таких как глобальные ресурсы, средства коммуникаций, предоставляемых операционной системой.

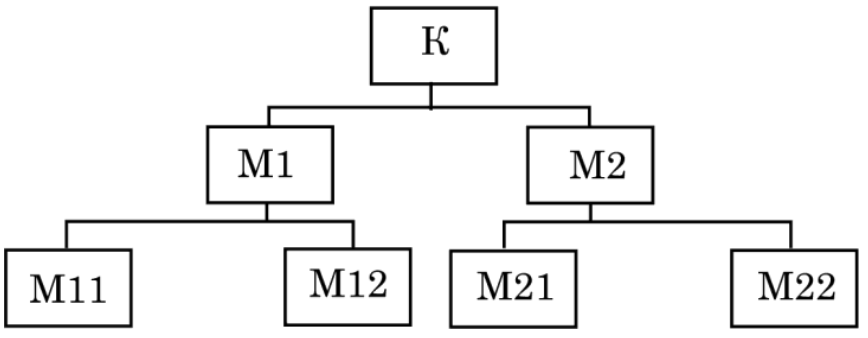


Рис. 2.1
Пример структуры комплекса программ

На рисунке 2.1 приведена структура комплекса программ K, состоящего из оттестированных на этапе модульного тестирования модулей M1, M2, M11, M12, M21, M22. Задача, решаемая методом интеграционного тестирования, — тестирование межмодульных связей, реализующихся при исполнении программного обеспечения комплекса K. Интеграционное тестирование использует модель «белого ящика» на модульном уровне. Поскольку тестировщику текст программы известен с детальностью до вызова всех модулей, входящих в тестируемый комплекс, применение структурных критериев на данном этапе возможно и оправдано.

Интеграционное тестирование применяется на этапе сборки модульно оттестированных модулей в единый комплекс. Известны два метода сборки модулей:

1) монолитный, характеризующийся одновременным объединением всех модулей в тестируемый комплекс;

2) инкрементальный, характеризующийся пошаговым (помодульным) наращиванием комплекса программ с пошаговым тестированием собираемого комплекса. В инкрементальном методе выделяют две стратегии добавления модулей:

- «сверху вниз» и соответствующее ему нисходящее тестирование;
- «снизу вверх» и, соответственно, восходящее тестирование.

Практическое задание

1. Создать приложение «Калькулятор», поддерживающее следующие функции:

- базовые функции (сложение, вычитание, умножение, деление);
- возведение в степень;
- извлечение квадратного корня;
- факториал выбранного числа;
- операции с памятью (сохранение числа в память, чтение из памяти, добавление к числу в памяти, очистка памяти).

2. Каждую операцию написать в виде отдельной функции.

3. Покрыть каждую из операций модульными тестами.

4. Убедиться, что тесты успешно проходятся в созданном приложении.

Содержание отчёта:

1. Цель работы.
2. Код приложения, включая код всех тестов.
3. Выводы по работе.

Контрольные вопросы

1. Дайте определение модульного тестирования.
2. Какова цель модульного тестирования?
3. По какому принципу проводится модульное тестирование?
4. Что включает в себя среда, создаваемая вокруг каждого модуля?
5. Какие дефекты проще всего обнаружить на уровне модульного тестирования?
6. Какие ошибки обычно пропускаются на уровне модульного тестирования и выявляются на более поздних стадиях тестирования?
7. Что определяет стратегию модульного тестирования?
8. Дайте определение интеграционного тестирования.
9. Почему с технологической точки зрения интеграционное тестирование является количественным развитием модульного?
10. В чём заключается основная разница между модульным и интеграционным тестированием?
11. На каком этапе применяется интеграционное тестирование?
12. Охарактеризуйте известные методы сборки модулей.